Operating Systems

Lecture 2 — OS Principles

Study Guide: What to Memorize & What to Understand

✓ MEMORIZE — Facts, definitions, numbers you must recall	★ UNDERSTAND — Concepts, trade-offs, why things work

1. What is an Operating System?

MEMORIZE	An OS is (1) a program that controls execution of application programs, and (2) an interface between applications and hardware.

MEMORIZE	Three OS objectives: Convenience, Efficiency, Ability to Evolve.

UNDERSTAND	The OS hides ugly hardware details (device controllers, interrupts, memory buses) behind a clean, uniform API that apps can use without knowing hardware specifics.

Layers of a Computer System

- Hardware → Operating System → Utilities/Libraries → Application Programs
- Three viewpoints: End User (sees apps), Programmer (sees API), OS Designer (sees hardware)

Key Interfaces (MEMORIZE these names)

Interface Boundaries		
Interface	**What it separates**	**Example**
Application Programming Interface (API)	App ↔ Libraries/OS	POSIX, Win32
Application Binary Interface (ABI)	Libraries ↔ OS	System call ABI
Instruction Set Architecture (ISA)	OS ↔ Hardware	x86-64, ARM

2. OS Services

What the OS provides (MEMORIZE the categories)

- Program development — editors, debuggers, frameworks
- Program execution — loads instructions + data, initializes I/O, schedules
- I/O device access — uniform interface hiding hardware details
- File access — authorization, sharing, protection, caching
- System access — protection, conflict resolution
- Error detection & response — hardware errors (memory, device failure) and software errors (arithmetic, forbidden memory access)
- Accounting — collect stats for performance monitoring and billing

OS as a Resource Manager

UNDERSTAND	The OS is itself a set of programs that runs like ordinary software. The key difference: it directs the processor's use of resources and willingly gives up the CPU to run other programs. It is built around the kernel (nucleus) — the portion always resident in main memory containing the most frequently used functions.

MEMORIZE	Resources managed by the OS: I/O devices, Memory, Processors.

3. Modes of Operation

MEMORIZE	Two modes: User Mode and Kernel Mode (also called Monitor Mode).

User Mode vs Kernel Mode		
	User Mode	**Kernel Mode**
Who runs here?	User programs / applications	OS kernel (monitor)
Memory access	Only allowed regions (protected)	Full access to all memory
Privileged instructions	NOT executable — causes trap	Executable
Purpose	Run user code safely	Manage hardware and resources

UNDERSTAND	Protection motivation: multiple programs sharing memory and CPU need isolation. Without modes, a buggy app could overwrite the OS or another app. The mode bit in the CPU enforces this split at hardware level.

4. Multiprogramming & Time Sharing

Why Multiprogramming?

MEMORIZE CPU utilization problem example: Read file (15μs) + Execute (1μs) + Write (15μs) = 31μs total. CPU busy only 1/31 = 3.2% of the time! I/O is ~30× slower than CPU.

Multiprogramming (MEMORIZE mechanism)

- Multiple processes reside in main memory simultaneously
- OS picks one job and runs it
- When the running job blocks on I/O → CPU switches to another job
- CPU is never idle as long as there are ready jobs

UNDERSTAND Uniprogramming: CPU sits idle waiting for every I/O. Multiprogramming: CPU switches to another ready job during any I/O wait — dramatically improving utilization.

Multiprogramming Gains (MEMORIZE numbers from example)

Uniprogramming vs Multiprogramming (3 jobs)		
Metric	**Uniprogramming**	**Multiprogramming**
Processor use	22%	43%
Memory use	30%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min	15 min
Throughput	6 jobs/hr	12 jobs/hr
Avg response	18 min	10 min

Time Sharing

MEMORIZE Time Sharing = extension of multiprogramming for interactive jobs. CPU switches between tasks by a time slice / Quantum. Each switch preempts the current user and loads the next.

MEMORIZE CTSS (1962): First time-sharing system. 32K 36-bit words, switched every 0.2 sec, supported up to 32 users.

Batch Multiprogramming vs Time Sharing		
	Batch Multiprogramming	**Time Sharing**

Principal objective	Maximize processor use	Minimize response time
Source of directives	Job control language with job	Commands at terminal
Scheduling emphasis	Throughput	Fairness / responsiveness

Time Sharing Problems (understand these)

- Memory protection — multiple jobs in memory must not interfere
- File system protection — only authorized users can access
- Resource contention — printers, storage must be managed

5. Processes

Definition (MEMORIZE all four)

A Process is defined as...
Definition
A program in execution
An instance of a running program
The entity that can be assigned to and executed on a processor
A unit of activity characterized by a single sequential thread, a current state, and associated system resources

Process Components (MEMORIZE)

MEMORIZE	A process has 3 components: (1) Executable program, (2) Associated data (variables, workspace, buffers, stacks), (3) Execution context — internal OS data: PC, registers, priority, I/O state.

UNDERSTAND	The execution context (process state) is how the OS can suspend and resume a process. It captures the entire register file, PC, priority, and I/O wait information — everything needed to restart exactly where it stopped.

Process Context Example (UNDERSTAND)

- Process index register → points to current running process entry in process list
- Program counter (PC) → next instruction to execute
- Base & Limit registers → define the memory region owned by this process
- Context switch → OS saves context of running process, restores context of next

Why Processes Were Needed — The 3 Problems (UNDERSTAND)

- Improper synchronization — bad signal design causes lost or duplicate signals
- Failed mutual exclusion — two processes using same shared resource simultaneously (e.g., editing same file)
- Non-deterministic operation — interleaved execution overwrites shared memory unpredictably
- Deadlocks — two processes each waiting for the other to release a resource

UNDERSTAND	The key tool that made processes possible: the interrupt. It suspends the running process, saves context (PC, registers), branches to the interrupt handler, then resumes another job. Without interrupts, the CPU could never switch safely.

6. Memory Management

5 Responsibilities (MEMORIZE)

- Process isolation — prevent interference between data and instructions
- Automatic allocation & management — transparent to user
- Support for modular programming — dynamic process creation/destruction
- Protection & access control — sharing with safety
- Long-term storage — file system

Virtual Memory (UNDERSTAND)

UNDERSTAND	Programs use logical (virtual) addresses independent of physical memory limits. The Memory Management Unit (MMU) hardware translates virtual → real addresses at runtime. If a page is not in RAM, it is fetched from disk (page fault) and the faulting process is suspended temporarily.

Paging (MEMORIZE mechanism)

- Process is divided into fixed-size blocks called pages
- Virtual address = page number + offset within page
- Main memory is divided into fixed-size frames (same size as pages)
- A page can be placed in any available frame — no contiguous allocation required
- If referenced page is not in RAM → OS fetches it from disk

UNDERSTAND	Paging eliminates external fragmentation (any page fits any frame) and enables virtual memory larger than physical RAM by keeping only active pages in memory.

7. Scheduling & Resource Management

3 Scheduling Policy Factors (MEMORIZE)

- Fairness — equal and fair access to resources
- Differential responsiveness — discriminate between job classes (e.g., interactive vs batch)
- Efficiency — maximize throughput, minimize response time

3 Types of Queues (MEMORIZE)

OS Scheduling Queues		
Queue	**Contains**	**Scheduler Action**
Short-term queue	Processes in/partially in RAM, ready to run	Short-term scheduler picks next (Round Robin, Priority)
Long-term queue	New jobs waiting to enter system	OS admits job, allocates RAM, moves to short-term queue
I/O queue (per device)	Processes waiting for a specific I/O device	OS assigns process to available device

UNDERSTAND	OS must not over-commit memory or CPU. Admitting too many jobs causes thrashing (constant swapping). The long-term scheduler controls the degree of multiprogramming.

Data Protection (MEMORIZE the 4 dimensions)

- Availability — protect against interruption of service
- Confidentiality — unauthorized users cannot read data
- Data integrity — protect data from unauthorized modification
- Authenticity — verify identity of users and validity of messages

8. Multithreading & Multiprocessing

Thread vs Process		
	Thread	**Process**
Unit type	Dispatchable unit of work	Collection of 1+ threads + resources
Has its own	PC, stack pointer, stack data area	Memory (code+data), open files, devices
Execution	Sequential, interruptible	Contains threads that execute
Benefit	Fine-grained concurrency within one app	Isolation and resource ownership

MEMORIZE

Multiprogramming: only one process executes at a time on one CPU (interleaved). Multiprocessing (SMP): multiple processes run truly simultaneously, each on a different CPU.

UNDERSTAND

Multithreading and SMP are independent concepts that complement each other. Multithreading is useful even on a single CPU (overlapping I/O with computation). SMP is useful even for non-threaded processes (running different processes in parallel).

9. OS Architecture Designs

Monolithic Kernel

MEMORIZE	All OS services (scheduling, FS, networking, device drivers, memory) run in a single large process sharing one address space in kernel mode.

Microkernel (UNDERSTAND why it's better)

MEMORIZE	Microkernel contains only: IPC (Inter-Process Communication) and basic scheduling. Everything else (device drivers, file system, VM manager, windowing, security) runs as user-mode server processes.

Microkernel Benefits (MEMORIZE the list)

- Uniform interface — no distinction between kernel-level and user-level services
- Extensibility — add new services without modifying kernel
- Flexibility — add or remove features easily
- Portability — only microkernel needs changing for new hardware
- Reliability — modular design enables rigorous testing; a crashed server doesn't crash the kernel
- Distributed system support — microkernel talks to local and remote servers the same way

Monolithic vs Layered vs Microkernel

	Monolithic	Layered	Microkernel
Kernel size	Large	Medium	Tiny
Performance	Best (direct calls)	Moderate	Overhead from IPC
Reliability	Bug crashes kernel	Better	Best (user-mode servers)
Example	Linux (modular monolithic)	Windows NT	macOS (Mach-based)

10. Virtualization & Cloud

Virtualization (UNDERSTAND)

UNDERSTAND	A single physical machine runs multiple VMs, each behaving as if it has its own hardware and OS. The Virtual Machine Manager (VMM / Hypervisor) mediates access to real hardware. Enables running incompatible OS versions simultaneously, testing, and data-center consolidation.

MEMORIZE	Emulation: source CPU type differs from target (e.g., PowerPC → x86). Every instruction must be translated — slowest. Interpretation: bytecode executed by interpreter. Virtualization: guest OS natively compiled for same CPU type as host — fastest.

Cloud Service Models (MEMORIZE)

Cloud Service Models		
Model	**What you get**	**Example**
IaaS — Infrastructure as a Service	Servers, storage, networking	AWS EC2, S3
PaaS — Platform as a Service	Software stack (DB, runtime)	AWS RDS, Heroku
SaaS — Software as a Service	Full application over Internet	Gmail, Office 365

11. Fault Tolerance

Key Metrics (MEMORIZE)

MEMORIZE	R(t) = probability of correct operation up to time t. MTTF = Mean Time To Failure. MTTR = Mean Time To Repair. Availability = fraction of time system is available = MTTF / (MTTF + MTTR).

Availability Classes (MEMORIZE)

Availability Classes		
Class	**Availability**	**Annual Downtime**
Continuous	1.0 (100%)	0 minutes
Fault Tolerant	0.99999	5 minutes
Fault Resilient	0.9999	53 minutes
High Availability	0.999	8.3 hours
Normal Availability	0.99–0.995	44–87 hours

Fault Categories (MEMORIZE)

- Permanent — always present until component is replaced/repaired
- Transient — occurs only once
- Intermittent — occurs at multiple unpredictable times

Redundancy Methods (MEMORIZE all three)

Methods of Redundancy		
Type	**Mechanism**	**Effective Against**
Spatial (Physical)	Multiple components perform same function simultaneously or as hot standby	Permanent & intermittent faults
Temporal	Repeat the operation when an error is detected	Transient/temporary faults only (NOT permanent)
Information	Replicate or encode data so bit errors can be detected and corrected	Data corruption, bit errors

12. Real OS Examples

UNIX / Linux

- Hardware surrounded by kernel, kernel isolates users and apps
- Traditional UNIX: two subsystems — File system and Process control subsystem
- Linux: modular monolithic kernel — not a microkernel, but uses loadable kernel modules (LKMs)
- LKMs: dynamically linked/unlinked at runtime; stackable with defined dependencies

Windows Architecture (MEMORIZE kernel-mode components)

Windows Kernel-Mode Components	
Component	**Role**
Executive	Core services: memory mgmt, process/thread mgmt, security, I/O, IPC
Kernel	Scheduling, context switching, synchronization
HAL (Hardware Abstraction Layer)	Isolates OS from platform-specific hardware differences
Device Drivers	Translate user I/O calls into hardware device requests
Windowing & Graphics	Implements GUI (GDI/Win32 USER)

Android

- Linux-based OS originally for mobile; first commercial release: Android 1.0 (2008)
- Architecture layers from bottom: Linux Kernel → HAL → Android Runtime (ART/Dalvik) → System Libraries → Application Framework → Applications
- Open Source (AOSP) — key to its widespread adoption

Quick-Recall Cheat Sheet

Numbers to Memorize

Key Numbers	
Fact	**Value**
CPU util with R+E+W (15+1+15 μs)	1/31 = 3.2%
Multiprogramming speedup (3-job example)	2× throughput, ½ elapsed time
CTSS year / quantum / users	1962 / 0.2 sec / 32 users
Fault Tolerant availability	0.99999 = 5 min downtime/year
Android 1.0 release year	2008

One-Line Definitions to Memorize

- Kernel: portion of OS always in main memory; most frequently used functions
- Process: program in execution with context, data, and executable
- Execution context: PC + registers + priority + I/O state = everything to suspend/resume a process
- Virtual memory: programs use logical addresses; MMU maps to physical at runtime
- Paging: process split into fixed pages; placed in any free frame
- Time sharing: multiprogramming + preemptive time quanta for interactive users
- Microkernel: tiny kernel (IPC + scheduling); all else runs as user-mode servers
- Multithreading: process subdivided into threads sharing memory but with own PC/stack
- MTTF: avg time between failures; MTTR: avg time to repair

Good luck, Gewily — you've got this.